

WANDERLY

Travel SaaS Platform Architecture & Build Plan

Trip management • Search & booking • AI itinerary planning

Prepared for: **Ahmed**

Document: v1.0 • Draft



1. Executive Summary

Wanderly is a SaaS travel platform that consolidates three pain points into one product: managing existing travel (bookings, itineraries, documents), searching and booking new travel across flights / hotels / attractions, and generating intelligent itineraries with an AI agent. The platform monetizes through a freemium subscription and booking commissions from travel providers.

Why this stack works

Next.js 14 covers the user-facing surface — auth, dashboards, server actions, and a thin BFF — letting you ship a polished web app fast.

A dedicated Go microservice handles search aggregation where the workload is fundamentally different: fan-out to 4–6 providers per query, aggressive caching, strict latency budgets. Go's concurrency model (goroutines + context cancellation) is ideal here and keeps cloud bills sane.

Claude is used as a tool-using agent for itinerary generation rather than a chatbot — it calls the same internal search APIs the user does, so itineraries are always grounded in real, bookable inventory.

Key numbers (MVP target)

Metric	Target	Notes
Time to MVP	12 weeks	1 backend, 1 full-stack, part-time design
Cloud infra cost	\$280–\$450 / mo	AWS or Hetzner + Cloudflare; scales to ~5k MAU
Provider API cost	\$0.01–\$0.04 / search	Mostly Amadeus + Duffel; cached aggressively
Break-even	~600 paid users	Or ~\$80k/yr in booking commissions
Target gross margin	70%+	After provider fees, infra, payment processing

Contents

1. Executive Summary	2
2. Product Vision & Personas	4
3. Feature Map	5
4. System Architecture	6
5. Tech Stack & Rationale	8
6. Data Model (Prisma schema)	10
7. Search & Booking Service (Go)	13
8. Provider Integrations	15
9. AI Itinerary Agent	17
10. API Contracts	20
11. Booking Flow	22
12. Infrastructure & Deployment	23
13. Security & Compliance	25
14. Observability	26
15. 12-Week Sprint Plan	27
16. Cost Model	29
17. Risks & Mitigations	30
18. Next Steps	31

2. Product Vision & Personas

Wanderly is built around the insight that most travelers juggle 4–6 different apps and inboxes for a single trip — flights in one app, hotels in another, attractions on a blog, confirmation emails scattered across Gmail. The product collapses this into one timeline, then layers booking and AI planning on top so users never need to leave.

Primary personas

Persona	Profile	Core jobs-to-be-done
The Frequent Business Traveler	30–50, takes 12+ trips/yr, employer reimbursed but plans on own	Centralize confirmations; fast rebooking when plans shift; expense-friendly receipts
The Aspirational Leisure Planner	25–45, plans 2–3 big trips/yr, spends weeks researching	AI-assisted itineraries; price tracking; visualizing the trip; sharing with travel partner
The Family Trip Coordinator	30–55, plans for spouse + 2–4 kids, complex needs	Multi-passenger booking; activity-fit for kids; document storage; group budget
The Digital Nomad	22–40, 6+ months/year traveling, lives across destinations	Long-stay hotels/apartments; visa tracking; recurring expense reporting

Differentiation vs incumbents

Competitor	What they do well	Where Wanderly wins
Triplt	Email parsing, timeline view	Adds search/booking + AI planning + group features
Booking.com	Hotel inventory & price	Trip-centric (not transaction-centric); flights + activities together
Hopper	Flight prediction + bundles	Full trip context, not just point-in-time deals
Wanderlog	Itinerary planning + maps	Real bookable inventory, not just ideas
ChatGPT / Bard	Conversational planning	Tool-grounded itineraries that turn into actual bookings

3. Feature Map

Roadmap is organized into three releases. MVP focuses on the core loop (plan → book → manage), while V2 and V3 expand into adjacent revenue streams and platform reach.

MVP (12 wks)	V2 (Q2)	V3 (Q3+)
• Auth + trip CRUD • Manual booking entry • Email-based booking import • Flight + hotel search (Amadeus) • Stripe payments + commission • AI itinerary generator • Trip sharing (read-only)	• Attractions (Viator/GYG) • Price drop alerts • Group trips & cost split • Mobile PWA + offline • Multi-currency • Loyalty points tracking	• Native iOS/Android • B2B agency tier • Affiliate marketplace • Carbon offset • Travel insurance add-on • Visa & docs assistant

MVP non-goals (intentionally excluded)

- Cars / rail / ferry — adds complexity without proving the core hypothesis.
- Native mobile apps — PWA is sufficient for v1; native ships in V3 once retention is proven.
- B2B / agency tier — would dilute product focus; revisit once consumer PMF is established.
- Insurance, eSIMs, lounge access — high-margin but deferred to avoid scope creep.

4. System Architecture

The platform follows a **modular monolith on the user-facing path** and a **specialized microservice for search aggregation**. This hybrid avoids the operational tax of full microservices on day one while isolating the one workload (multi-provider search) that genuinely benefits from a separate runtime.

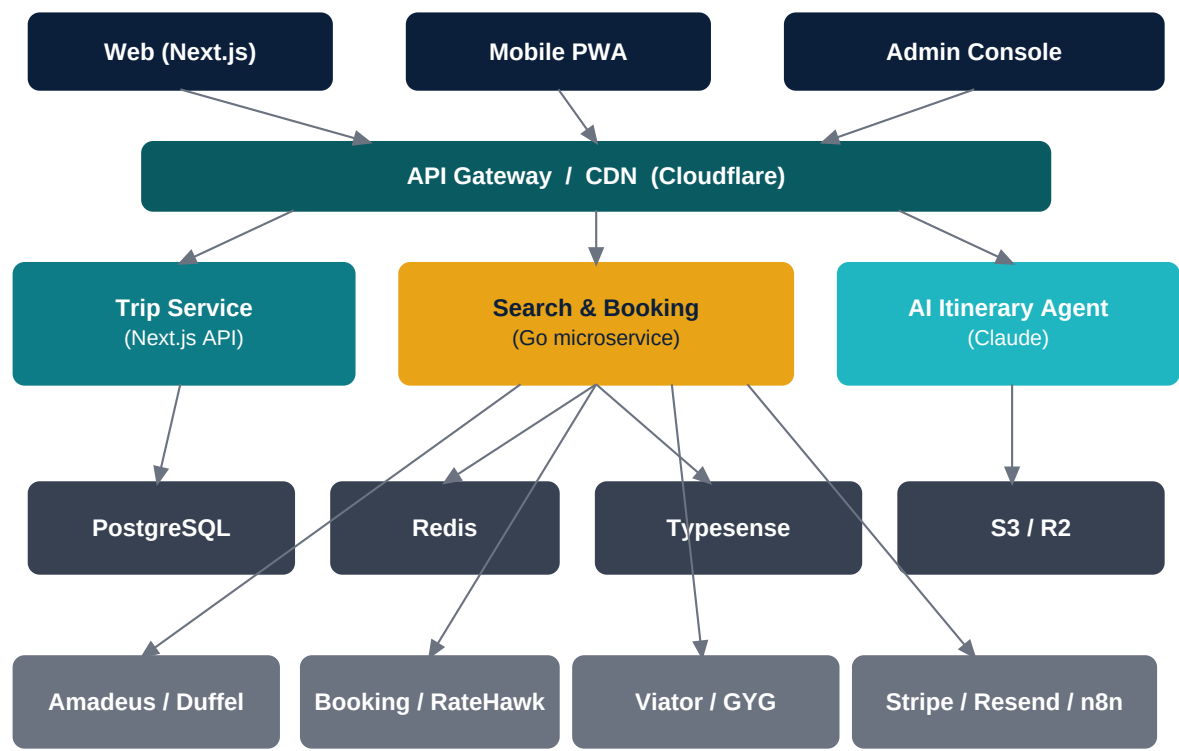


Figure 1 — High-level component view. Solid arrows indicate request flow; data tier and external providers are shaded.

Bounded contexts

Context	Responsibility
Trip Management	Owns trips, segments (flights, lodging, activities), travelers, documents, sharing. Backed by the main Postgres schema. Implements the timeline, sharing, and import-from-email flows.
Search & Booking	Owns the Go aggregator service + provider adapters. Caches quotes in Redis with provider-specific TTLs (flights: 5–15 min, hotels: 30 min). Owns idempotent booking, payment intent creation, and webhook reconciliation.
AI Planning	Owns the Claude agent runtime, tool definitions, and conversation state. The agent is given the same internal APIs as the UI — it cannot hallucinate inventory because it has to actually call search.

Context	Responsibility
Identity & Billing	Auth (NextAuth + Postgres), subscription state via Stripe Customer Portal, plan gating middleware.
Notifications & Workflows	n8n hosts background workflows: booking confirmations, price-drop alerts, post-trip surveys, abandoned-cart recovery.

Cross-cutting decisions

Communication. Synchronous HTTP between Trip ↔ Search; async events (Redis Streams or NATS) for booking confirmations, email parsing, AI background tasks. No Kafka in MVP — overkill for traffic levels.

Idempotency. All booking POSTs require an Idempotency-Key header. Stored in Redis with 24h TTL keyed to userId+key.

Caching. Two layers: CDN (Cloudflare) for static + page shells; Redis for search quotes and rate limits. Postgres is never hit on a search miss path.

Multi-tenancy. Single-tenant schema with userId/orgId on every row. Row-level security enforced at Prisma middleware level. Defer real multi-tenant DB until B2B (V3).

Internationalization. next-intl from day one; English + Arabic at launch given target geography. RTL layout tested in CI.

5. Tech Stack & Rationale

Choices below favor proven, well-documented tools over novelty. Each decision lists why it was picked and what was rejected.

Layer	Choice	Why	Rejected
Frontend	Next.js 14 (App Router) + TypeScript + Tailwind + shadcn/ui	RSC + server actions cut boilerplate; Tailwind ships fast UIs; mature ecosystem	Remix (smaller community), SvelteKit (less library coverage)
Backend (web)	Next.js Route Handlers + Prisma	Same repo, same types end-to-end; Prisma's type safety eliminates a class of bugs	NestJS (heavier), tRPC (locks into TS-only client)
Backend (search)	Go 1.22 + chi router + sqlc	Goroutines for fan-out; tiny container; predictable memory; sub-50ms p50 budget feasible	Node (worse concurrency), Rust (slower iteration), Python (GIL)
Database	PostgreSQL 16 (managed: Neon or RDS)	JSONB for flexible itinerary docs; rock-solid; row-level security for sharing	MongoDB (worse for relational trip↔segment data)
Cache	Redis 7 (Upstash for serverless or self-hosted)	Quote caching, rate limits, idempotency keys, session store	Memcached (no streams), DynamoDB (cost at low scale)
Search	Typesense (self-hosted)	Fast typo-tolerant search for destinations + POIs; cheaper than Algolia at scale	Algolia (pricing), Elasticsearch (operational tax)
AI	Claude Sonnet 4 + Anthropic SDK + tool use	Best tool-use reliability for agentic flows; long context for trip docs	GPT-4o (also good; second source for redundancy)
Payments	Stripe (Subscriptions + PaymentIntents + Connect)	Connect lets you collect commission cleanly; mature for travel use case	Adyen (overkill at this stage)
Email	Resend + React Email	Best-in-class DX; React templates match the codebase	Postmark, SES (more setup)
Workflows	n8n (self-hosted)	You already use it; non-eng team can build alerts/marketing flows	Zapier (per-task pricing), Temporal (overkill)
Maps	Mapbox GL JS + Mapbox Tiles	Better customization than Google Maps; predictable pricing	Google Maps (cost at scale)
Observability	OpenTelemetry → Grafana Cloud (traces + logs + metrics)	Vendor-neutral; Grafana free tier is generous	Datadog (expensive), New Relic (similar)
CI/CD	GitHub Actions + Docker + Fly.io (or Hetzner + Coolify)	Fly's global edge fits the user base; Coolify if cost is dominant concern	AWS ECS (more YAML), Vercel (lock-in)

Layer	Choice	Why	Rejected
Auth	Auth.js v5 + Postgres adapter + Passkeys	Passkeys are the right default in 2026; Auth.js stays close to Next.js	Clerk/Auth0 (cost), Supabase Auth (less flexible)

6. Data Model (Prisma)

The data model centers on a **Trip** aggregate that owns a heterogeneous list of **Segments** (flights, lodgings, activities, transfers) plus shared resources (travelers, documents). Bookings sit alongside segments to capture the commerce side independently from the planning side — this lets users add wishlist items without committing to a booking.

```
// Prisma schema (excerpt) – datasource & generators omitted

model User {
  id          String    @id @default(cuid())
  email       String    @unique
  name        String?
  passkeys    Passkey[]
  trips       Trip[]
  bookings    Booking[]
  subscription Subscription?
  createdAt   DateTime @default(now())
}

model Trip {
  id          String    @id @default(cuid())
  ownerId     String
  owner       User      @relation(fields: [ownerId], references: [id])
  title       String
  destination String      // primary destination, "Tokyo, JP"
  startDate   DateTime
  endDate     DateTime
  status       TripStatus @default(PLANNING) // PLANNING | BOOKED | ONGOING | COMPLETED
  budgetCents Int?
  currency     String    @default("USD")
  travelers    Traveler[]
  segments     Segment[]
  bookings     Booking[]
  documents    Document[]
  shares       TripShare[]
  metadata     Json       @default("{}")
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  @@index([ownerId, startDate])
}

model Segment {
  id          String    @id @default(cuid())
  tripId      String
  trip        Trip      @relation(fields: [tripId], references: [id], onDelete: Cascade)
  type        SegmentType // FLIGHT | LODGING | ACTIVITY | TRANSFER | NOTE
  startsAt    DateTime
  endsAt      DateTime?
  title       String
  payload     Json       // type-specific structured data
  bookingId   String?
  booking     Booking?   @relation(fields: [bookingId], references: [id])
  orderHint   Int        @default(0)

  @@index([tripId, startsAt])
}
```

Polymorphic segment payloads are stored as JSONB and validated at the application layer with Zod schemas keyed off type. This avoids a sprawling table-per-type structure while keeping type safety end-to-end.

```

model Booking {
  id          String    @id @default(cuid())
  userId      String
  tripId      String?
  provider     String    // "amadeus" | "duffel" | "ratehawk" | ...
  providerRef  String    // PNR, confirmation number, etc.
  status      BookingStatus // QUOTED | PENDING | CONFIRMED | CANCELLED | FAILED
  totalCents  Int
  commissionCents Int    @default(0)
  currency    String
  rawOffer    Json      // verbatim provider response, audit trail
  stripePiId  String?   @unique
  idempotencyKey String @unique
  createdAt   DateTime @default(now())
  confirmedAt  DateTime?

  @@index([userId, createdAt])
  @@index([provider, providerRef])
}

model Subscription {
  id          String    @id @default(cuid())
  userId      String    @unique
  stripeCustomerId String @unique
  stripeSubId String?   @unique
  plan        Plan      @default(FREE) // FREE | PRO | TEAM
  currentPeriodEnd DateTime?
  cancelAtPeriodEnd Boolean @default(false)
}

model TripShare {
  id          String    @id @default(cuid())
  tripId      String
  trip        Trip      @relation(fields: [tripId], references: [id], onDelete: Cascade)
  email       String?
  token       String    @unique      // share-by-link
  role        ShareRole @default(VIEWER) // VIEWER | EDITOR
  expiresAt   DateTime?
}

model Document {
  id          String    @id @default(cuid())
  tripId      String
  type        DocType   // PASSPORT | VISA | TICKET | RECEIPT | OTHER
  s3Key       String
  filename    String
  parsed      Json?     // extracted fields when applicable
  createdAt   DateTime @default(now())
}

```

Indexing strategy

- **Trip:** composite (ownerId, startDate) for the primary dashboard query.
- **Segment:** (tripId, startsAt) drives the timeline view.
- **Booking:** (provider, providerRef) for webhook reconciliation; (userId, createdAt) for booking history.
- **JSONB payloads:** GIN indexes on Segment.payload and Booking.rawOffer for ad-hoc analytics queries.

7. Search & Booking Service (Go)

Provider search is the one workload that materially benefits from being a separate Go service. A flight search hits 4–6 providers in parallel, each with 2–8 second latency budgets, and the user perceives whichever is slowest. Goroutines + context deadlines make this clean to implement and easy to reason about.

Service responsibilities

- **Provider adapters:** one Go package per provider. Each implements a small interface (Search, QuoteOffer, Confirm, Cancel).
- **Aggregator:** fans out to all enabled providers, merges results, normalizes schemas, dedupes by route+price.
- **Ranker:** sorts by user-selected criteria (cheapest, fastest, best-value composite).
- **Cache:** Redis hash keyed by query hash → quote list. Provider-specific TTL.
- **Booking executor:** confirms an offer, creates the Stripe PaymentIntent, persists Booking row, fires webhook.
- **Reconciler:** hourly job that fetches provider order status for any in-flight bookings.

Provider adapter interface (Go)

```
// internal/providers/provider.go
package providers

import (
    "context"
    "time"
)

type FlightQuery struct {
    Origin      string    // IATA, "JFK"
    Destination string    // IATA, "DXB"
    Depart      time.Time
    Return      *time.Time
    PaxAdult    int
    PaxChild    int
    Cabin       string    // "economy" | "business" | ...
}

type Offer struct {
    ID            string          // provider-scoped offer id
    Provider      string
    TotalCents    int
    Currency      string
    Slices        []Slice
    BaggageRules  []BaggageRule
    Refundable    bool
    ExpiresAt     time.Time      // CRITICAL: offers expire
    Raw           json.RawMessage // verbatim payload (for audit)
}

type Provider interface {
    Name() string
    SearchFlights(ctx context.Context, q FlightQuery) ([]Offer, error)
    QuoteOffer(ctx context.Context, offerID string) (*Offer, error)
}
```

```
    Confirm(ctx context.Context, offerID string, pax []Passenger,
            payment PaymentRef) (*ConfirmedBooking, error)
    Cancel(ctx context.Context, bookingRef string) error
}
```

Aggregator (concurrent fan-out)

```
// internal/aggregator/flights.go
func (a *Aggregator) SearchFlights(
    ctx context.Context, q providers.FlightQuery,
) ([]providers.Offer, error) {
    // Hard deadline so a slow provider can't block UX
    ctx, cancel := context.WithTimeout(ctx, 6*time.Second)
    defer cancel()

    // Cache check
    key := flightCacheKey(q)
    if cached, ok := a.cache.Get(ctx, key); ok {
        return cached, nil
    }

    type result struct{ offers []providers.Offer; err error; provider string }
    out := make(chan result, len(a.providers))

    for _, p := range a.providers {
        p := p
        go func() {
            offers, err := p.SearchFlights(ctx, q)
            out <- result{offers, err, p.Name()}
        }()
    }

    var all []providers.Offer
    for i := 0; i < len(a.providers); i++ {
        r := <-out
        if r.err != nil {
            a.metrics.ProviderError(r.provider, r.err)
            continue // soft fail; partial results are fine
        }
        all = append(all, r.offers...)
    }

    all = dedupe(all)
    rank(all, q) // composite scoring
    a.cache.Set(ctx, key, all, 10*time.Minute)
    return all, nil
}
```

8. Provider Integrations

Provider selection optimizes for: (a) developer experience, (b) inventory breadth, (c) commission economics, (d) availability in target geographies. The plan starts narrow (one primary per category) and adds redundancy as volume justifies it.

Provider	Category	Why	Commission	Notes
Amadeus Self-Service	Flights + Hotels	Best DX; sandbox + production; single contract for both	Tiered, ~3–8%	Primary for MVP. Test → Prod gating.
Duffel	Flights	Modern API; great error messages; airline NDC content	~2–5%	Secondary; redundancy + better NDC fares.
Booking.com Affiliate	Hotels	Largest hotel inventory; trusted brand for end users	Up to 4%	Apply early — review process is slow.
RateHawk / TBO	Hotels (B2B)	Net rates with markup margin instead of commission split	Variable (you set markup)	Better economics at scale; more KYC required.
Viator (Tripadvisor)	Activities	Largest activity inventory; affiliate program is mature	8%	Easy approval; deep-linking option for MVP if API takes time.
GetYourGuide	Activities	Curated supply; better UX for some destinations	Up to 8%	Apply alongside Viator.
Google Places	POI / enrichment	Photos, hours, ratings — not for booking	n/a	Used to enrich destination pages.
Mapbox	Maps + geocoding	Custom styles; predictable pricing	n/a	Free tier comfortably covers MVP.

Integration playbook

For each provider, the integration follows the same five-step pattern:

- 1. Apply early.** Affiliate/partner approval often takes 1–4 weeks — start before you need them.
- 2. Sandbox first.** Build adapter against the test endpoint; record fixtures for unit tests.
- 3. Schema normalize.** Map provider response → internal Offer struct. Lossy fields go into Raw.
- 4. Reconciliation job.** Hourly cron pulls order status for in-flight bookings; reconciles drift.
- 5. Kill switch.** Feature-flag every provider. A misbehaving provider can be removed from rotation in seconds.

Compliance gotcha

Travel agent licensing requirements vary by jurisdiction. In Saudi Arabia (likely a target market given context), the Ministry of Tourism requires a license for travel agency operations. Two paths: (a) operate under a licensed partner's umbrella initially, (b) obtain a Class A or Class B agency license (~3–6 months). EU markets require Package Travel Directive compliance (insolvency protection bond). Plan for both before V2.

9. AI Itinerary Agent

The itinerary agent is built as a **tool-using Claude agent**, not a chatbot wrapping a model. It receives a brief (destination, dates, budget, preferences), then iteratively calls the same internal search APIs the user uses — flights, hotels, activities, places. The output is a structured itinerary JSON that the UI renders directly into the trip timeline.

Why tool use, not free-text

- **Grounding.** The agent can't recommend a flight that doesn't exist or a hotel that's fully booked.
- **Bookability.** Every recommendation includes a real offer ID that goes straight to checkout.
- **Pricing accuracy.** Prices come from live provider quotes, not training data.
- **Auditability.** Every tool call is logged — full trace of why each item ended up in the itinerary.

Tool definitions

```
// Tools exposed to the agent. Same shapes as the public API.
const tools = [
  {
    name: "search_flights",
    description: "Search flights between two cities. Returns ranked offers.",
    input_schema: {
      type: "object",
      required: ["origin", "destination", "departDate"],
      properties: {
        origin: { type: "string", description: "City or IATA code" },
        destination: { type: "string", description: "City or IATA code" },
        departDate: { type: "string", format: "date" },
        returnDate: { type: "string", format: "date" },
        paxAdult: { type: "integer", default: 1 },
        cabin: { type: "string", enum: ["economy", "premium", "business", "first"] }
      }
    }
  },
  {
    name: "search_hotels",
    description: "Search hotels in a city for given dates.",
    input_schema: { /* city, checkIn, checkOut, guests, minStars, maxNightlyCents */ }
  },
  {
    name: "search_activities",
    description: "Search bookable activities/attractions in a city.",
    input_schema: { /* city, date, categories[], maxPriceCents */ }
  },
  {
    name: "get_destination_info",
    description: "Lookup weather, visa requirements, currency, language for a destination.",
    input_schema: { /* destination, monthIso */ }
  },
  {
    name: "save_itinerary",
    description: "Persist the constructed itinerary to the user's trip.",
    input_schema: { /* tripId, segments[] */ }
  }
];
```

System prompt (abridged)


```
You are an itinerary planning agent for Wanderly. Your job is to build a
day-by-day itinerary for a user given their brief.

Rules:
1. ALWAYS call search_* tools before recommending anything. Never invent flights,
   hotels, or activities from memory.
2. Respect the user's budget. Track running cost; if a recommendation would push
   total over budget, downgrade an earlier item or flag the conflict explicitly.
3. Build a coherent geography: don't put two activities 4 hours apart on the
   same morning. Cluster by neighborhood when possible.
4. For each day, produce: (1) the lodging anchor, (2) 1-3 activities, (3) at
   most one travel-day note. Leave breathing room – over-planning ruins trips.
5. When done, call save_itinerary with the structured plan. Do NOT just print it.
6. If the user's brief is contradictory (e.g., "luxury Tokyo trip for $300"),
   surface the contradiction in the final reply and ask for guidance.

Cost discipline:
- Cache aggressively. Re-call search_flights only if dates or cities changed.
- Limit to ~12 tool calls per planning session unless the user asks for revisions.

Output style:
- Final reply to the user is a short summary (3-5 sentences), then the structured
  itinerary is rendered by the UI from save_itinerary's payload.
```

Cost & latency budget

Item	Per planning session	Notes
Claude tokens (in)	~6–15k	System prompt + tool results
Claude tokens (out)	~2–5k	Reasoning + tool calls + final summary
Tool calls	~6–12	search_flights, search_hotels, search_activities × cities
Wall clock	20–60s	Streamed UI keeps it tolerable
Cost per session	~\$0.08–\$0.25	Gate behind Pro plan to align cost with revenue

Anti-pattern to avoid

Don't expose the agent as an open chat. Constrain it to the planning task with a clear start and end — brief in, itinerary out. Open chats invite jailbreaks, drift off-topic, and balloon token costs without improving outcomes. The chat affordance comes back in V2 as a 'revise' button on each segment.

10. API Contracts

REST + JSON. OpenAPI 3.1 spec lives in the repo and generates both the TypeScript client (for the Next.js app) and the Go client (for the search service to call back into Trip Service). Below are the most important endpoints.

Method	Path	Purpose
POST	/api/v1/trips	Create a trip
GET	/api/v1/trips	List user's trips (paginated, filterable by status)
GET	/api/v1/trips/{id}	Trip detail with segments + bookings
POST	/api/v1/trips/{id}/segments	Add a manual segment (note, ad-hoc booking)
POST	/api/v1/trips/{id}/share	Create a share token
POST	/api/v1/search/flights	Search flights (proxies to Go service)
POST	/api/v1/search/hotels	Search hotels
POST	/api/v1/search/activities	Search activities
POST	/api/v1/bookings	Create booking from offer (idempotent)
GET	/api/v1/bookings/{id}	Booking status + provider ref
POST	/api/v1/bookings/{id}/cancel	Cancel + refund per provider rules
POST	/api/v1/itinerary/plan	Kick off AI planner (returns task id)
GET	/api/v1/itinerary/plan/{taskId}	Stream planner progress (SSE)
POST	/api/v1/imports/email	Receive forwarded confirmation emails for parsing
POST	/api/v1/webhooks/stripe	Stripe webhook (subscriptions + payments)
POST	/api/v1/webhooks/{provider}	Provider booking status webhooks

Sample request / response

```
POST /api/v1/search/flights
Authorization: Bearer <jwt>
Idempotency-Key: client-uuid-v4
Content-Type: application/json
```

```
{
  "origin": "RUH",
  "destination": "NRT",
  "departDate": "2026-08-12",
  "returnDate": "2026-08-26",
  "paxAdult": 2,
  "cabin": "economy"
}
```

```
200 OK
```

```
{
  "queryId": "qry_01HZX...",
  "ttlSeconds": 600,
  "offers": [
    {
      "offerId": "off_amad_8sj...",
      "provider": "amadeus",
      "totalCents": 421500,
      "currency": "USD",
      "slices": [ /* 0D legs */ ],
      "baggage": { "checked": 1, "carryOn": 1 },
      "refundable": false,
      "expiresAt": "2026-05-06T14:32:11Z"
    }
  ]
}
```

Error model

All errors return RFC 7807 Problem Details JSON with stable type URIs. Provider failures surface as 502 Bad Gateway with a provider field so the client can show a partial-result banner instead of an error toast.

11. Booking Flow

The booking flow is the riskiest path in the system. Money moves, third parties can fail, offers expire mid-checkout. The design below makes the flow idempotent and recoverable at every step.

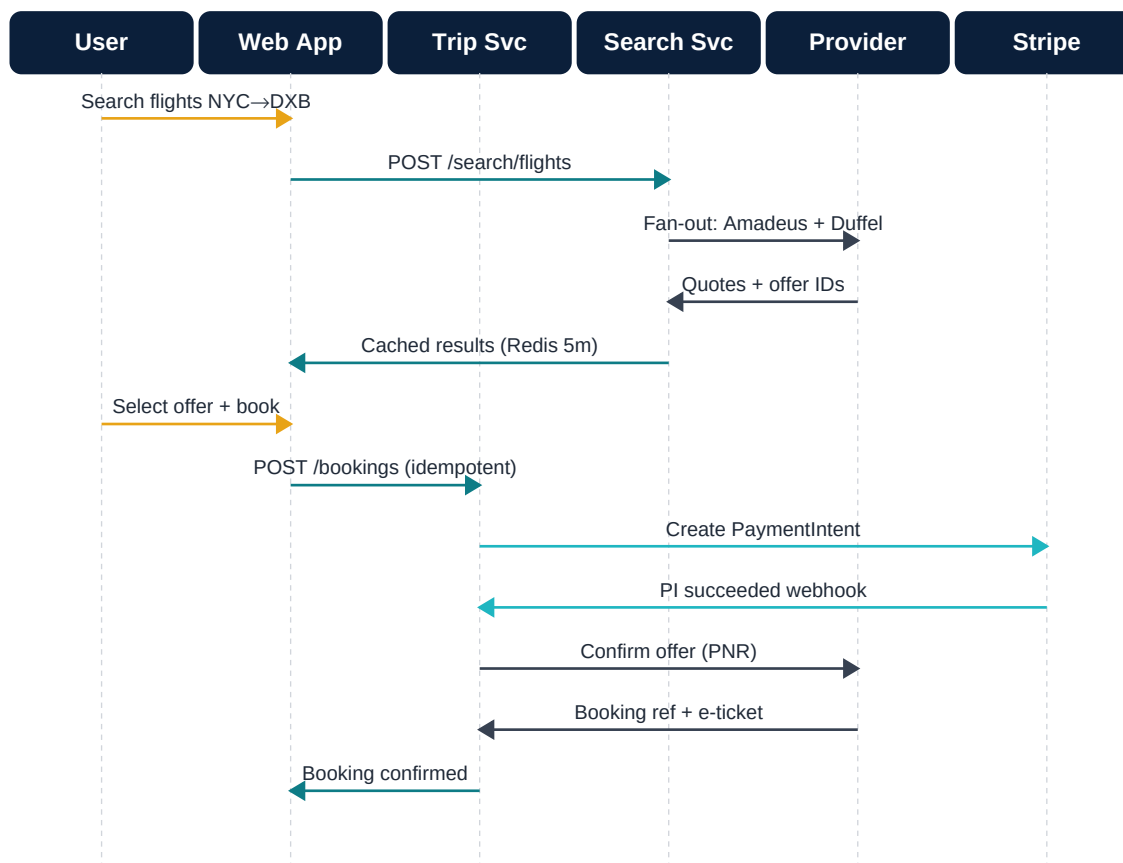


Figure 2 — Booking sequence (happy path).

Failure modes & recovery

Failure	Detection	Recovery
Offer expired before confirm	Provider returns OFFER_EXPIRED	Re-quote silently; if price within 5% delta, auto-confirm; else surface to user.
Stripe charge succeeds, provider confirm fails	Confirm step throws after PI captured	Auto-refund via Stripe; mark booking FAILED; email user; never leave money in limbo.
Stripe charge fails, provider already held inventory	PI fails; provider hold may be active	Cancel provider hold; release inventory; user retries with different card.
Webhook arrives before sync confirm response	Race condition on status update	Idempotent state machine: only forward transitions allowed; out-of-order events drop.
User double-clicks 'Book'	Two POSTs with same Idempotency-Key	Second request returns first request's response from Redis.

12. Infrastructure & Deployment

Two viable hosting paths depending on cost-vs-effort preference. Both reach the same end state.

Component	Path A (Fly.io / Vercel)	Path B (Hetzner + Coolify)
Web app	Vercel (Next.js)	Coolify on Hetzner CPX31
Search service (Go)	Fly.io app, 3 regions	Coolify, 2 nodes
Postgres	Neon (serverless, branchable)	Self-hosted Postgres + pgBackRest
Redis	Upstash (serverless)	Self-hosted Redis
Object storage	Cloudflare R2	MinIO on Hetzner
CDN / WAF	Cloudflare	Cloudflare
CI/CD	GitHub Actions → Vercel/Fly	GitHub Actions → Coolify webhooks
~Monthly cost @ MVP	\$280–\$450	\$80–\$140
Time to set up	~2 days	~1 week
Best for	Speed to market, lean team	Cost-sensitive scale, ops-comfortable team

Environments

- **local** — docker-compose with all dependencies; seeded fixtures.
- **preview** — auto-deploy per PR; ephemeral Neon branch + ephemeral Redis namespace.
- **staging** — long-lived; uses provider sandboxes; mirrors prod config.
- **production** — live providers; Stripe live mode; restricted access to logs/secrets.

Deployment pipeline

GitHub Actions runs: lint → typecheck → unit tests → integration tests (with provider VCR fixtures) → container build → trivy scan → deploy to preview → smoke tests → manual promote to staging → manual promote to prod. Search service has a separate canary stage: 5% traffic for 30 minutes before full rollout.

13. Security & Compliance

Area	Control
Authentication	Auth.js v5; Passkeys preferred; email magic-link fallback; TOTP for high-value accounts.
Authorization	Row-level security via Prisma middleware; every query scoped to userId/orgId by default.
Secrets	Doppler or Infisical; no .env in repo; rotation playbook for provider keys.
PII	Encrypted at rest (Postgres TDE / volume encryption); PII fields tagged in schema for export/delete.
Payments	Stripe Elements — PAN never touches our servers; PCI SAQ-A scope only.
Travel docs	Passport scans encrypted with per-user envelope keys (KMS); auto-purge 30 days post-trip.
GDPR / Saudi PDPL	Data export + delete endpoints from day one; retention policy documented; DPA with all subprocessors.
Rate limits	Per-IP + per-user buckets in Redis; tighter limits on /search and /auth.
Bot defense	Cloudflare Turnstile on signup, login, and password reset.
Audit logs	Append-only audit table for booking, payment, and PII-access events.
Vulnerability mgmt	Dependabot + Trivy in CI; quarterly pen test once revenue justifies it.
Incident response	Documented runbook; on-call rotation; status page (Statuspage or self-hosted).

Saudi-specific note

Saudi PDPL (effective 2024) requires data of Saudi residents to be stored in-Kingdom unless explicit consent is captured for cross-border transfer. Plan for a regional Postgres replica in AWS me-south-1 or STC Cloud before V2 launch in KSA.

14. Observability

OpenTelemetry from day one. Three pillars route to Grafana Cloud (free tier). The single biggest operational risk in a travel platform is a misbehaving provider — telemetry is built around making that visible in seconds.

Signal	Tooling	Key dashboards
Traces	OTel SDK → Tempo (Grafana Cloud)	End-to-end booking trace; provider call latency breakdown
Metrics	Prometheus exporters → Mimir	Provider error rate; search p50/p95/p99; booking conversion funnel
Logs	Pino (Node) / zap (Go) → Loki	Structured JSON; correlation_id linking across services
Errors	Sentry	Frontend + backend; release tracking + source maps
Synthetic	Checkly	Booking flow monitor; runs every 15 min in staging + prod
Real user	Grafana Faro / Cloudflare Web Analytics	Web Vitals, regional breakdown

Critical SLOs (MVP)

- **Search latency p95** ≤ 3.5s (with provider fan-out).
- **Booking confirmation success** ≥ 98.5% (excludes user payment failures).
- **Page TTI p75** ≤ 2.5s on mobile 4G.
- **Webhook processing** ≤ 30s from provider event to user notification.
- **Availability** ≥ 99.5% for booking endpoints (allows ~3.5h/mo of unplanned downtime).

15. 12-Week Sprint Plan

Six 2-week sprints. Assumes one full-stack engineer + one part-time backend engineer + part-time design. Each sprint ends with a deployed, demoable build behind a feature flag.

Sprint	Theme	Deliverables	Dependencies
1 (wks 1–2)	Foundations	Repo, CI/CD, auth (passkeys + magic link), trip CRUD, basic timeline UI, design system from shadcn/ui, OTel scaffolding	Domain, Cloudflare, hosting account
2 (wks 3–4)	Manual entry + email import	Add segments manually; forward-to-import email parser (regex + LLM hybrid); document upload to R2; share-by-link	Resend domain verified; R2 bucket
3 (wks 5–6)	Flight + hotel search	Go search service skeleton; Amadeus adapter; aggregator + Redis cache; Next.js search UI	Amadeus sandbox keys (apply day 1)
4 (wks 7–8)	Booking + payments	Booking endpoint (idempotent); Stripe Connect onboarding; payment intent flow; webhook reconciliation; commission tracking	Stripe live keys; Amadeus prod approval
5 (wks 9–10)	AI itinerary agent	Agent runtime; tool definitions; planning UI with SSE streaming; save-to-trip flow; Pro plan gating	Stripe Subscriptions live
6 (wks 11–12)	Polish + soft launch	Mobile PWA pass; multi-currency display; price-drop alert (n8n); seed content for top 20 destinations; closed beta launch	Beta waitlist (~200 users)

Definition of Done (every sprint)

- All new code has tests; coverage doesn't drop below 70% on backend, 50% on frontend.
- OpenAPI spec updated and clients regenerated.
- OTel spans on every new endpoint.
- Feature flag in place; rollback path documented.
- Demo recorded (Loom) and posted in #product channel.

16. Cost Model

Costs at three scale points. 'MVP' assumes ~500 MAU / 2k searches per day. 'Growth' assumes ~10k MAU / 40k searches per day. 'Scale' assumes ~50k MAU / 200k searches per day.

Line item	MVP	Growth	Scale
Hosting (web + Go + n8n)	\$60	\$320	\$1,400
Postgres (Neon / managed)	\$25	\$160	\$600
Redis (Upstash / managed)	\$10	\$80	\$280
Cloudflare (CDN + R2)	\$20	\$80	\$220
Mapbox	Free tier	\$120	\$650
Resend (email)	\$20	\$90	\$300
Stripe (% of GMV)	—	—	—
Claude API (planner sessions)	\$40	\$320	\$1,500
Provider API quotas	Included	\$200	\$900
Observability (Grafana Cloud)	Free tier	\$50	\$300
Sentry	\$26	\$80	\$240
Misc (Doppler, status page, domains)	\$25	\$60	\$150
TOTAL / month	~\$226	~\$1,560	~\$6,540

Revenue model

- **Free tier:** 3 trips, manual entry only, no AI planner. Drives signups + email parsing data.
- **Pro (\$9/mo or \$79/yr):** Unlimited trips, AI planner, price alerts, group sharing.
- **Booking commission:** 3–8% per booking (varies by provider). Average target: 5%.
- **Affiliate:** Insurance, eSIM, lounge passes (V3) at 10–25% commission.

Unit economics (target)

Metric	Target	Notes
CAC (paid)	\$18–\$30	Mostly content + paid social
ARPU (Pro)	\$78/yr	After Stripe fees
GMV per Pro user / yr	\$2,800	Conservative — 1 trip booked through platform
Commission per Pro user / yr	\$140	5% of GMV

Metric	Target	Notes
Total ARPU (Pro)	~\$218/yr	Subscription + commission
LTV (24mo expected life)	~\$436	Discounted
LTV / CAC	~17x	Excellent if achievable

17. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Amadeus prod approval delayed	Medium	High	Apply in week 1; Duffel as parallel track; sandbox demo if delayed
Provider commission rates lower than modeled	Medium	High	Diversify across 2+ providers per category; price-test Pro plan upward; layer in net-rate (RateHawk) earlier
Travel agent licensing required in launch market	High	Medium	Operate under licensed partner umbrella for first 6 months; pursue own license in parallel
AI planner quality below user expectations	Medium	Medium	Hand-curate seed content for top 20 destinations; A/B against pure-LLM baseline; gate behind Pro to manage expectations
Stripe holds funds (chargebacks)	Low	High	Strict KYC on signup; clear refund/cancel UX; disputes runbook from day one
Single-engineer key-person risk	Medium	High	Document everything; pair-program key flows; keep stack mainstream (no exotic tools)
Provider outage during peak booking	High	Medium	Multi-provider per category; circuit breakers; cached results; status page
Currency / FX volatility on commissions	Low	Medium	Settle in USD; daily FX rate refresh; hedge if exposure exceeds \$50k/mo
GDPR / PDPL data request handling	Medium	Medium	Build export/delete from day one; document retention; DPA with all subprocessors
Competitor (Booking, Hopper) clones AI planner	High	Low	Speed of execution; integrated experience is the moat, not the model

18. Next Steps

Concrete actions for the next 14 days, in priority order:

#	Action	Owner	Output
1	Apply for Amadeus Self-Service prod access	You	API keys (sandbox immediate, prod 1–4 wks)
2	Apply for Booking.com Affiliate + Viator partner	You	Approval emails
3	Reserve domain + set up Cloudflare zone	You	DNS pointing to placeholder
4	Spin up Neon project + Upstash Redis + R2 bucket	You	Connection strings in Doppler
5	Initialize monorepo (apps/web, apps/search, packages/db, packages/sdk)	Eng	Repo skeleton in GitHub
6	Decide on Path A vs Path B hosting (Fly+Vercel vs Hetzner+Coolify)	You	Decision logged in ADR-001
7	Brand/name lock + basic landing page	Design	Live wait-list page
8	Run small user research: 5 calls with target personas	You	Validates persona job priorities; informs MVP scope
9	Stripe account + Connect onboarding	You	Test mode account active
10	Set up Sentry, Grafana Cloud, Doppler workspaces	Eng	Free-tier accounts wired in

One thing to build this week

Before any code, do the 5 user research calls (action #8). The riskiest assumption in this plan isn't technical — it's that travelers will pay \$9/mo for a planning tool when they currently use 6 free apps. Five 30-min calls will either validate the painkiller framing or point you to a sharper one before \$50k of build cost lands on the wrong product.

Open questions for follow-up

- Target launch geography — KSA-first, GCC, MENA-wide, or global from day one?
- Brand identity — name, tone, visual direction (the doc uses 'Wanderly' as a placeholder)
- Capital availability — bootstrapped, friends & family, or seed round? Affects pace and hiring.
- Co-founder situation — solo, technical co-founder, full team?
- Existing audience or distribution — newsletter, social following, B2B partnerships to leverage?

End of document. Generated as a starting architecture — every decision here is up for revision once user research and provider conversations sharpen the picture.